DjOpenKB

DjOpenKB is a Docker-based Django knowledge base platform that integrates article management, user-submitted knowledge workflows, Active Directory authentication, local fallback login, MFA enforcement, Vault-backed secret management, PostgreSQL persistence, Nginx HTTPS reverse proxying, and an integrated OpenKB AI/chatbox workflow.

The application is designed for an internal knowledge repository where users can browse articles, suggest new content, search existing knowledge, and use the OpenKB AI assistant after the knowledge base has been synced into the OpenKB data store.

Main Features

- **Knowledge base website** built with Django.
- **Article browsing and search** for Markdown-based knowledge content.
- **User article suggestion workflow** with draft, pending review, approved, and pending-failed states.
- **Admin review tools** for approving, editing, returning, or rejecting suggested articles with feedback.
- **AD/LDAP login** as the main domain authentication flow.
- **Local Django login fallback** for local administrator or fallback accounts.
- **MFA enforcement** using authenticator-app TOTP codes as part of the login completion flow.
- **Vault secret management** for Django, PostgreSQL, LDAP bind password, and AI provider API keys.
- **PostgreSQL persistence** using a bind-mounted `postgres-data/` runtime folder.
- **Nginx HTTPS reverse proxy** for serving the website over TLS.
- **Integrated OpenKB AI/chatbox** using the downloaded OpenKB source from VectifyAI/OpenKB.
- **Multilingual interface support** through Django locale files.
- **Cleanup scheduler** for removing stray uploaded files.

Documentation

Use these documents together:

File	Purpose
`README.md`	Project overview, features, folder structure, and dependency summary.
`DEPLOYMENT_GUIDE.md`	Full first-time setup, normal startup, update, troubleshooting, Vault, AD/LDAP, MFA, and OpenKB commands.

For deployment, start with:

```
cat DEPLOYMENT_GUIDE.md
```

Project Structure

```
DjOpenKB/
├── djopenkb/                    # Django project settings, URLs, ASGI/WSGI entry
points
├── kb/                          # Main Django app: articles, auth, MFA, admin
tools, search, profiles
├── website/templates/           # HTML templates for login, profile, articles,
admin pages, MFA pages
├── website/static/              # Static frontend assets used by the website
├── locale/                      # Django translation files for supported
languages
├── docker/                      # Container helper scripts, including Postgres
Vault entrypoint
├── nginx/                       # Nginx reverse proxy config and HTTPS
certificate files/scripts
├── vault/                       # Vault configuration, bootstrap, scripts, file
storage, and runtime keys
├── openkb-data/                 # Local OpenKB workspace/data folder,
generated/runtime content
├── OpenKB-main/                 # Downloaded OpenKB source from
https://github.com/VectifyAI/OpenKB
├── postgres-data/               # PostgreSQL persistent runtime data
├── staticfiles/                 # Collected static files generated by Django
├── docker-compose.yml           # Main Docker Compose service definition
├── Dockerfile                   # Django web container build file
├── Dockerfile.postgres-vault    # PostgreSQL image that reads password from Vault
├── requirements.txt             # Python package dependencies for the Django web
container
├── manage.py                    # Django management command entry point
└── DEPLOYMENT_GUIDE.md          # Setup, operation, troubleshooting, and
maintenance commands
```

Core Folders

`djopenkb/`

Contains the Django project configuration:

- `settings.py` – application settings, installed apps, middleware, database, LDAP, Vault, static files, and security settings.
- `urls.py` – root URL routing for the project.
- `asgi.py` / `wsgi.py` – server entry points.

`kb/`

Main Django application folder. It contains:

- article models and suggestion workflow logic
- authentication backends for LDAP/local login
- MFA helper logic and middleware

- views for articles, search, profile, admin tools, review pages, and MFA pages
- Django admin registration
- management commands such as OpenKB sync and LDAP testing

`website/templates/`

HTML templates used by the site. Important pages include:

- login page
- profile page
- article display and article editor pages
- pending article review pages
- MFA setup and verification pages
- general layout templates

`locale/`

Django translation files:

```
locale/<language>/LC_MESSAGES/django.po
locale/<language>/LC_MESSAGES/django.mo
```

Update `.po` files when adding/changing text, then compile messages when required.

Docker and Runtime Services

DjOpenKB runs through Docker Compose. Main services include:

Service	Purpose
`vault`	Stores application secrets.
`vault-init`	Initializes/unseals/seeds Vault during setup or secret rotation.
`vault-auto-unseal`	Auto-unseals Vault for local VM deployment.
`db`	PostgreSQL database. Reads `POSTGRES_PASSWORD` from Vault.
`web`	Django/Gunicorn application container.
`nginx`	HTTPS reverse proxy in front of Django.
`cleanup-scheduler`	Periodic stray upload cleanup task.

Vault and Secret Management

Secrets are not intended to live in the normal `.env` file. The normal `.env` should contain non-secret configuration, while `vault/bootstrap/djopenkb.env` is used only to seed Vault.

Secrets stored in Vault include:

```
DJANGO_SECRET_KEY
POSTGRES_PASSWORD
LDAP_BIND_PASSWORD
GEMINI_API_KEY
LLM_API_KEY
OPENAI_API_KEY, if using OpenAI
```

Runtime Vault folders:

```
vault/bootstrap/       # Temporary bootstrap secret file location
vault/file/            # Vault persistent data
vault/keys/            # Vault unseal/app token material
vault/logs/            # Vault logs
```

Important notes:

- `vault/bootstrap/djopenkb.env` is temporary and should not be committed.
- After Vault is seeded, remove `vault/bootstrap/djopenkb.env`.
- Do not change `POSTGRES_PASSWORD` after PostgreSQL is already initialized unless you also update the password inside PostgreSQL.
- Do not push real contents from `vault/file/`, `vault/keys/`, or `postgres-data/`.

Authentication and MFA

DjOpenKB supports two login paths:

1. **AD/LDAP login** for domain users.
2. **Local Django login** for local fallback accounts.

MFA is treated as part of the login completion flow. A password-authenticated user is not considered fully logged in until MFA setup or MFA verification is completed.

General MFA behavior:

```
Username/password accepted
→ MFA setup required if no authenticator exists
→ MFA verification required if authenticator already exists
→ only after valid TOTP code is the session fully authenticated
```

Users cannot access protected DjOpenKB pages until MFA is complete.

OpenKB AI Integration

DjOpenKB integrates the OpenKB source downloaded from:

```
https://github.com/VectifyAI/OpenKB
```

Key folders:

```
OpenKB-main/       # Downloaded OpenKB source code
openkb-data/       # Local OpenKB workspace and generated knowledge-base data
```

OpenKB must be initialized before syncing data into the AI workflow. During `openkb init`, select the AI provider/model that matches the deployment, such as Gemini, OpenAI, or another supported provider.

After initialization, DjOpenKB articles can be synced into OpenKB using:

```
docker compose exec web python manage.py sync_openkb_ai
```

Make sure the chosen provider's API key is stored in Vault through `vault/bootstrap/djopenkb.env` during first setup, or patched into Vault later.

Runtime Folders Not to Commit

The following folders/files are required at runtime but should not be committed with real data:

```
.env
vault/bootstrap/djopenkb.env
vault/file/
vault/keys/
vault/logs/
postgres-data/
openkb-data/
staticfiles/
```

Use `.gitkeep` files to keep empty folder structure in GitHub while ignoring runtime contents.

Python Requirements

The main Python dependencies are listed in `requirements.txt`.

Package	Purpose
`Django`	Main web framework.
`gunicorn`	Production WSGI server for the Django app.
`Markdown`	Markdown article rendering.

Package	Purpose
`bleach`	HTML sanitization for rendered content.
`Pillow`	Image handling for uploaded/static image processing.
`python-dotenv`	Loading environment-style configuration.
`django-auth-ldap`	AD/LDAP authentication integration.
`psycopg[binary]`	PostgreSQL database driver.
`pyotp`	TOTP authenticator-code generation and verification.
`qrcode[pil]`	QR code generation for authenticator app setup.

Install/update dependencies by rebuilding the Docker web image:

```
docker compose up -d --build --force-recreate web nginx
```

Quick Start

Use the full deployment instructions in `DEPLOYMENT_GUIDE.md`.

At a high level:

```
# Clone the repository
git clone https://github.com/ErinFlyingSkyRocket/DjOpenKB.git
cd DjOpenKB

# Prepare runtime folders and config files
mkdir -p vault/bootstrap vault/file vault/keys vault/logs postgres-data openkb-
data/raw openkb-data/wiki
cp .env.example .env
cp vault/bootstrap/djopenkb.env.example vault/bootstrap/djopenkb.env

# Edit deployment values and secrets
nano .env
nano vault/bootstrap/djopenkb.env

# Start the stack
docker compose up -d --build

# Run migrations and create admin account
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser

# Initialize and sync OpenKB AI workflow
docker compose exec web sh -lc "cd /app/openkb-data && PYTHONPATH=/app/OpenKB-main
python -m openkb.cli init"
docker compose exec web python manage.py sync_openkb_ai
```

```
# Remove plaintext bootstrap secrets after Vault is seeded
rm -f vault/bootstrap/djopenkb.env
```

For normal future startup:

```
docker compose up -d
```

Important Configuration Values

Before deployment, update these values for the target environment.

.env

```
DJANGO_DEBUG=false
DJANGO_ALLOWED_HOSTS=<SERVER-IP-OR-DOMAIN>,localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=https://<SERVER-IP-OR-
DOMAIN>:8080,https://localhost:8080,https://127.0.0.1:8080

LDAP_ENABLED=true
LDAP_SERVER_URI=ldap://<AD-SERVER-IP>:389
LDAP_BIND_DN=<SERVICE-ACCOUNT>@<AD-DOMAIN>
LDAP_AD_DOMAIN=<AD-DOMAIN>
LDAP_NETBIOS_DOMAIN=<NETBIOS-DOMAIN>
LDAP_ALLOWED_EMAIL_DOMAINS=<AD-DOMAIN>
LDAP_USER_SEARCH_BASE=DC=<DOMAIN-PART>,DC=<SUFFIX>

OPENKB_BASE_DIR=OpenKB-main
OPENKB_DATA_DIR=openkb-data
OPENKB_AI_PROVIDER=openkb-cli
OPENKB_GEMINI_MODEL=<OPENKB-MODEL-NAME>
```

vault/bootstrap/djopenkb.env

```
DJANGO_SECRET_KEY="<LONG-RANDOM-DJANGO-SECRET>"
POSTGRES_PASSWORD="<STABLE-POSTGRES-PASSWORD>"
LDAP_BIND_PASSWORD="<AD-SERVICE-ACCOUNT-PASSWORD>"
GEMINI_API_KEY="<GEMINI-API-KEY-IF-USING-GEMINI>"
LLM_API_KEY="<OPENKB-LITELLM-COMPATIBLE-KEY>"
OPENAI_API_KEY="<OPENAI-API-KEY-IF-USING-OPENAI>"
```

Maintenance

Common commands:

```
# View all logs
docker compose logs -f

# View web logs only
docker compose logs -f web

# Restart web and nginx after template/settings changes
docker compose up -d --build --force-recreate web nginx

# Run migrations
docker compose exec web python manage.py migrate

# Re-sync OpenKB AI after content changes
docker compose exec web python manage.py sync_openkb_ai
```

For full operations and troubleshooting commands, use:

```
cat DEPLOYMENT_GUIDE.md
```